

APA: Adaptive-Precision Attention

Exact FP16 for the Few Tiles That Matter, FP4 for the Rest

Raphael Friedmann

<https://github.com/kekz1/apa>

October 2026, APA 0.2.0

Abstract

In long-context causal prefill, the softmax mass of a query row concentrates in a small number of key tiles. APA is a causal flash-attention prefill kernel for the RTX 5090 (sm_120a) that exploits this: a first pass runs block-scaled FP4 attention (`mma.sync kind::mx4nxf4`) over every 64-key tile and flags a tile as *hot* when its share of the running row sum exceeds a threshold ε . Hot tiles skip the FP4 PV product and are recomputed by a second pass in exact FP16; the two partial results are merged by a log-sum-exp. Both passes run in one fused launch with persistent pass-2 workers. On Llama-3.2-3B attention dumps at 122880 keys, APA at $\varepsilon = 0.005$ flags 6.0 to 10.4% of active tiles as hot and runs in 4.506 to 4.668 ms, against 10.725 ms for the same kernel with every tile exact ($2.32\times$). Integrated into the imp inference engine, dense prefill of a 106451-token prompt drops from 11659.35 ms (FlashAttention-2) to 6433.23 ms ($1.81\times$), with perplexity below FlashAttention-2 with FP16 or FP32 accumulation on Llama-3.2-3B and Qwen3-8B. The kernel is deterministic (bitwise identical across reruns), supports GQA, paged KV caches and chunked prefill with an incremental tile cache, and is released as a header-only CUDA library under MPL-2.0.

1 Introduction

Prefill cost of long prompts is dominated by attention: for a prompt of length N the work is $O(N^2d)$ while the rest of the model is $O(Nd^2)$. FlashAttention [2, 1, 10] removes the memory bottleneck, but the arithmetic remains. Two families of methods attack it. *Low-precision* attention [16, 15, 14] computes $QK^\top$ and PV in INT8, FP8 or FP4 with smoothing and fine-grained scales; it keeps every key but adds error to every score. *Sparse* attention [12, 5, 11] skips keys predicted to carry little mass; it is exact on the kept keys and wrong on the skipped ones.

APA combines both at tile granularity and lets the data decide the split per query block. Every tile is computed cheaply in FP4, so no key is dropped. The few tiles that carry a large share of the row’s softmax mass, where FP4 error would dominate the output, are routed to an exact FP16 pass. The decision is made online inside the FP4

pass from quantities the pass computes anyway, so it costs one extra small MMA per tile.

Contributions:

- A two-pass attention scheme with an online hot-tile test whose cold verdicts are monotone safe: the running row sum only grows, so a tile judged cold stays below ε of the final row sum (Section 3.2).
- A pass-1 tile order (sink tile first, then the diagonal backwards) that puts high-mass tiles into the running sum early, so the test judges middle tiles against a near-final denominator.
- An FP4 pass on sm_120 block-scaled MMA with mean-centred K, per-16-key power-of-two P scales folded into the exponent, a lazy row maximum, and row sums obtained from the same quantized P via an MMA against a ones tile.
- A single fused launch: pass-1 CTAs followed by persistent pass-2 workers that pull work from ticket counters and wait on per-query-block release flags, deadlock-free by ticket order.
- Deterministic statistics and an incremental tile cache for chunked prefill over paged KV.

2 Background

Online softmax. For a query row q and keys k_j , flash attention keeps a running maximum m , row sum $\ell = \sum_j e^{s_j - m}$ and unnormalized output $o = \sum_j e^{s_j - m} v_j$, rescaling ℓ and o when m grows [7, 8]. Two partial states (m_1, ℓ_1, o_1) and (m_2, ℓ_2, o_2) over disjoint key sets merge exactly:

$$M = \max(m_1, m_2), \quad w_i = e^{m_i - M}, \quad O = \frac{w_1 o_1 + w_2 o_2}{w_1 \ell_1 + w_2 \ell_2}. \quad (1)$$

APA uses this to merge the FP4 result over cold tiles with the FP16 result over hot tiles. All exponentials run in the $\log_2$ domain (`ex2.approx`).

Step	Output
stats	per (batch, KV head): K column sums per chunk of 16 tiles, K and V absolute maxima
finalize	K mean $\bar{k}$ (chunks summed in order), head scales g_k, g_v
KV quant	per tile a 9216B blob: K (mean-centred) and $V^\top$ in E2M1, one UE4M3 scale per 16 values
Q quant	packed Q rows in E2M1 with UE4M3 scales; $q \cdot \bar{k}$ kept for pass 2

Table 1: Prep kernels. Blob = K 4096B, $V^\top$ 4096B, K scales 512B, V scales 512B, laid out for conflict-free `ldmatrix` (XOR swizzle) and for one bulk copy per tile.

Block-scaled FP4 on sm_120. The RTX 5090 executes block-scaled MMA (`mma.sync, kind::mxf4nvf4, m16n8k64`) with E2M1 operands and one UE4M3 scale per 16 elements along k (NVFP4, a variant of the OCP microscaling formats [9]). E2M1 represents $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$; UE4M3 has a maximum of 448. APA additionally uses a global FP32 scale per head so that $|x|_{\max}$ maps to $6 \cdot 448 = 2688$.

3 Method

APA computes causal attention for head dim $d = 128$ with GQA (H query heads, H_{kv} KV heads, $G = H/H_{kv}$). Keys are split into tiles of $B_{kv} = 64$. Query rows of the G heads sharing a KV head are *packed*: packed row r is position $\lfloor r/G \rfloor$ of query head $h_{kv}G + (r \bmod G)$, so one CTA of 12 warps covers $12 \cdot 16 = 192$ packed rows and loads each KV tile once for all G heads.

3.1 Preparation

A prep stage converts FP16 inputs into the FP4 layout (Table 1).

Mean-centred K. Subtracting the per-channel key mean, $s_j = q \cdot (k_j - \bar{k})$, shifts every score of a row by the same constant $q \cdot \bar{k}$ and leaves the softmax unchanged [16]. It removes the shared channel offset that would otherwise consume FP4 range. Pass 2 computes exact scores $q \cdot k_j$ from the original FP16 keys and subtracts $q \cdot \bar{k}$, so both passes share one logit frame and Eq. (1) applies.

3.2 Pass 1: FP4 attention with a hot-tile test

Each warp owns 16 packed rows; a CTA of 12 warps streams the tile blobs through a 6-stage shared-memory ring filled by `cp.async.bulk` with `mbarrier` completion (thread 0 is the producer). Per tile and warp:

1. $S \leftarrow \text{MMA}_{\text{fp4}}(Q, K_t)$; apply causal / tail mask
2. update lazy row max m (only if a score exceeds $m + \tau$)
3. per 16-key group: power-of-two scale 2^k , $P' \leftarrow 2^{sc+b}$ with $P' \leq 6$
4. $\ell_t \leftarrow \text{MMA}_{\text{fp4}}(P', \mathbf{1})$ tile row sums
5. $\lambda \leftarrow \lambda + \ell_t$ running sum, all tiles
6. if any row has $\ell_t > \varepsilon \lambda$: set bit t in the warp’s hot mask, stop
7. $\ell_{\text{cold}} \leftarrow \ell_{\text{cold}} + \ell_t$; $o \leftarrow o + \text{MMA}_{\text{fp4}}(P', V_t)$

Figure 1: Pass 1, one tile t of one warp.

P quantization. The probabilities of a 16-key group are scaled by $1536/2^k$ with $2^k \geq 256 \cdot \max_{\text{group}} P$, so the quantized values satisfy $P' \leq 6$ and the scale 2^k is an exact UE4M3 power of two. The shift is folded into the bias of the exponent, so `ex2` directly produces P' ; the global factor 1536 is divided out in the epilogue.

Lazy maximum. The running maximum m ($\log_2$ domain) is updated, with a full quad reduction and a rescale of o , ℓ_{cold} and λ , only when some score exceeds $m + \tau$ with $\tau = 4$. The bias absorbs the lag, so $P' \leq 6$ still holds; most tiles skip the rescale.

Row sums from the quantized P. Instead of summing FP32 probabilities, pass 1 multiplies the packed P' fragment by a tile of E2M1 ones. The resulting row sums are those of the values that enter PV , so numerator and denominator of the cold partial carry the same quantization.

The hot test. λ accumulates the sums of *all* tiles seen so far, hot ones included, while ℓ_{cold} and o accumulate only cold ones. A tile is hot for the warp if, for any of its 16 rows, its row sum exceeds $\varepsilon \lambda$. Since λ only grows over the KV loop (up to the common rescale), a cold tile’s share of the final row sum is at most its share at test time, so every cold verdict remains valid. A hot verdict can be conservative; it costs pass-2 work, not accuracy. The test is warp-uniform (`__any_sync`), so the hot mask has one bit per (warp, tile).

Tile order. The denominator λ is small early in the loop, which makes early tiles look hot. Pass 1 therefore visits tile 0 first (the attention sink [12]), then the remaining tiles from the causal diagonal backwards. The tiles that typically carry most mass enter λ first, and the long middle range is judged against a near-final sum.

Epilogue. A warp without hot tiles normalizes and stores its output directly. A warp with hot tiles exports (m, ℓ_{cold}) and the unnormalized cold partial o in FP32 for the merge.

3.3 Pass 2: exact FP16 over hot tiles

A pass-2 group of 4 warps takes the same packed rows as 4 consecutive pass-1 warps. It streams the union of their hot tiles from the original FP16 K and V (flat or paged), double-buffered with `cp.async`: K_{t+1} loads under PV_t , V_{t+1} under QK_{t+1} . Each warp computes only its own hot tiles with `mma.sync.m16n8k16` (f16 inputs, f16 accumulation, the default of imp’s FlashAttention-2 path) and an FP32 online softmax in the $\log_2$ domain. At the end each row is merged with its pass-1 partial by Eq. (1) and written to O .

3.4 Fused launch

Both passes run in one launch of $n_1 + n_{SM}$ CTAs of 384 threads, where n_1 is the number of (batch, KV head, query block) pass-1 items. Each CTA takes a ticket from an atomic counter. Tickets below n_1 run pass 1 (causal: heaviest query blocks first); the rest become persistent pass-2 workers whose three 4-warp groups pull (query block, sub-group) items from a second counter. A pass-2 group spins on its query block’s ready flag, which the pass-1 CTA sets with a release store after a fence; the worker reads it with acquire semantics. Tickets are handed to CTAs in the order they start running, so every pass-1 item a worker waits on is already resident: the scheme cannot deadlock. Pass-2 work fills the SMs left idle by pass 1’s last wave.

3.5 Determinism

All floating-point reductions run in a fixed order. In 0.1.0 the K column sums used shared and global float atomics; 5 reruns of prep and attention on the three dumps differed in 539072, 1329742 and 950802 output elements. In 0.2.0 each 16-tile chunk sums its columns in registers, then by XOR shuffles, then over warps in order, and the finalize kernel sums chunks in order. Maxima use integer `atomicMax`, which is order-independent. The same 5 reruns now give bitwise identical outputs; prep time fell from 0.865 to 0.731 ms at 122880 keys.

3.6 Paged KV and chunked prefill

Pass 2 reads K/V through a reader abstraction: flat [B] [S] [H_kv] [d] or a paged FP16 pool through a block table [6], plus a flat tail for the chunk not yet in the cache. For chunked prefill, `prefill_incremental` keeps a per-layer `KvState` with the quantized tile blobs and statistics; a new chunk quantizes only its new keys. Head scales are widened by a headroom factor so later keys fit, and a fingerprint of 48 key values detects a stale cache and triggers a full requantization.

Dump	cos mean	cos min	hot	attn ms	prep ms
lc_122880_0	0.999000	0.898838	10.4 %	4.506	0.730
lc_122880_1	0.999626	0.970893	9.7 %	4.615	0.731
lc_122880_2	0.997437	−0.444615	6.0 %	4.668	0.730

Table 2: Standalone, $\varepsilon = 0.005$, 2048 queries at 122880 keys. With every tile exact ($\varepsilon < 0$), lc_122880_1 takes 10.725 ms at cos mean 0.999634.

4 Evaluation

Setup. RTX 5090 (sm_120a), CUDA ≥ 12.9 . Standalone runs use attention inputs dumped from Llama-3.2-3B [4] (24 query heads, 8 KV heads, $d = 128$): 2048 query rows against 122880 keys, three dumps. Accuracy is the cosine similarity against an FP32 reference on 512 sampled (row, head) pairs; we report mean and minimum. Times are the minimum over 5 repetitions of the mean of 5 launches (CUDA events). Hot share is the fraction of active (warp, tile) pairs flagged hot.

Standalone. Table 2: at $\varepsilon = 0.005$ between 6.0 and 10.4 % of the active warp-tiles go to pass 2. Against the same code with every tile exact, APA is $2.32\times$ faster on lc_122880_1 (4.615 vs 10.725 ms) at a mean cosine of 0.999626 vs 0.999634. Prep adds 0.731 ms (prep and attention together: 5.346 ms).

End to end. APA is integrated into imp [3] behind `attention.apa_eps`. Llama-3.2-3B Q8_0, 106451-token prompt, `--max-seq-len 108000`, prefill time:

Mode	FA2 ms	APA ms	speedup
default (sparse prefill)	6661.95	5241.18	1.27 $\times$
dense	11659.35	6433.23	1.81 $\times$

Table 3: imp prefill, $\varepsilon = 0.005$. Dense: `sparse_prefill_topk_tokens=0`. Speedup covers the whole prefill, not only attention.

Perplexity. Table 4, imp on ppl_corpus_45k (13k to 14k tokens) for Llama-3.2-3B and Qwen3-8B [13] in Q8_0. The two FlashAttention-2 variants differ only in the accumulation type and span 17.3644 to 17.3732 (Llama) and 10.7974 to 10.8110 (Qwen). Every APA setting with $\varepsilon > 0$ lies below both; the all-exact run is 0.0009 above FA2 f32 on Llama. Raising ε from 0.002 to 0.01 cuts default-mode prefill from 5915.15 to 4863.56 ms. APA 0.2.0 reproduces the 0.1.0 values at $\varepsilon = 0.005$ exactly; the 0.002 and 0.01 rows were measured with 0.1.0.

5 Limitations

- **Worst-case rows.** On lc_122880_2 one sampled row reaches a cosine of -0.444615 while the mean

Variant	Llama 3B	Qwen3 8B	prefill ms
FA2, f16 acc.	17.3644	10.7974	6661.95
FA2, f32 acc.	17.3732	10.8110	
APA, all exact	17.3741	10.7868	
APA $\varepsilon = 0.002$	17.3556	10.7855	5915.15
APA $\varepsilon = 0.005$	17.3423	10.7869	5241.18
APA $\varepsilon = 0.01$	17.3385	10.7728	4863.56

Table 4: Perplexity (lower is better) and default-mode prefill time of the 106451-token prompt (Llama).

stays at 0.997437. The hot test bounds each cold tile’s share by ε , not the sum of many cold tiles’ FP4 errors. The cause of this row is not yet analysed. Perplexity does not show it; per-row guarantees need a stronger test, for example a bound on the accumulated cold mass.

- **Granularity.** The hot verdict is per (warp, tile): one row of 16 makes the tile exact for all.
- **Scope.** Head dim 128, causal, FP16 inputs, sm_120a only. Pass 2 of the fused kernel is specialised for $d = 128$.
- **Fixed ε .** One global threshold; a per-layer or per-head ε is not explored.
- **Baselines.** Speedups are against an all-exact run of the same kernel and against imp’s FlashAttention-2. A direct comparison with FP4 attention kernels such as SageAttention3 [14] on the same inputs is open.

6 Related work

FlashAttention [2, 1, 10] established IO-aware exact attention; FlashAttention-3 adds FP8 on Hopper. SageAttention [16, 15, 14] quantizes Q, K, P, V to INT8, INT4, FP8 and FP4 with K smoothing by mean subtraction, which APA adopts; it applies one precision to every tile. StreamingLLM [12] keeps sink and recent tokens, MInference [5] predicts sparse patterns for prefill, Quest [11] selects pages by query-aware bounds for decode. These skip keys; APA keeps every key and spends precision instead of dropping work, with the split decided online from the FP4 pass itself.

7 Conclusion

Long-context attention is skewed enough that exact arithmetic on 6 to 10 % of the tiles, FP4 on the rest, and an exact log-sum-exp merge reach the perplexity of FP16 attention at $2.32\times$ the speed of an all-exact kernel and $1.81\times$ dense prefill speedup end to end. The code is available at <https://github.com/kekz1/apa> under MPL-2.0.

References

- [1] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [3] Raphael Friedmann. imp: LLM inference engine. <https://github.com/kekz1/imp>, 2026.
- [4] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [5] Huiqiang Jiang et al. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*, 2023.
- [7] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- [8] Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.
- [9] Bitan Darvish Rouhani et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.
- [10] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [11] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.
- [12] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [13] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

- [14] Jintao Zhang et al. SageAttention3: Microscaling FP4 attention for inference and an exploration of 8-bit training. *arXiv preprint arXiv:2505.11594*, 2025.
- [15] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. In *International Conference on Machine Learning (ICML)*, 2025.
- [16] Jintao Zhang, Jia Wei, Pengle Zhang, Jun Zhu, and Jianfei Chen. SageAttention: Accurate 8-bit attention for plug-and-play inference acceleration. In *International Conference on Learning Representations (ICLR)*, 2025.